

ML-lab-report-d24b010

Kodali Ramana Gopal
DA24B010

November 2025

1 A-Summary of Models Used and System Architecture

Overview

The system classifies handwritten digits (0–9) from the MNIST dataset using the following pipeline. It combines three components:

- * PCA for dimensionality reduction
- * KNN classifier as the base learner
- * Softmax (multinomial logistic regression) model as the meta-learner.

This stacked design aims to improve stability, accuracy, and robustness.

Models Used

1. Principal Component Analysis (PCA) I used PCA to reduce the original 784 features to a smaller set (e.g., 100–150 components). This speeds up computation, removes noise, and makes KNN more effective in lower-dimensional space.

2. K-Nearest Neighbour (KNN) KNN classifies a sample by looking at the k closest training points using Euclidean distance. Distance-weighted voting is used so closer neighbours have more influence. KNN performs well on MNIST but can be slow and sensitive to noise, which is why it benefits from PCA.

3. Softmax Regression (Meta-Learner) The Softmax model takes KNN's probability outputs as input and learns to refine and correct its predictions. It provides smoother decision boundaries and better calibrated probabilities.

Stacked Architecture

The full system works as follows:

- Normalize raw images and apply PCA.
- KNN generates class probability vectors.
- Softmax uses these probability vectors to make the final prediction.

Out-of-fold predictions are used during training to avoid information leakage.

Why This Architecture is better than simple models.

- PCA reduces noise and speeds up KNN significantly.
- KNN captures local patterns in digit shapes.
- Softmax stabilizes KNN, reducing its sensitivity to noise.
- The stacked model improves reliability and reduces variance across runs.

Pros and Cons

Pros:

- Better generalization compared to using KNN or Softmax alone.
- Produces smoother and more reliable predictions.

- Handles confusing digit pairs (e.g., 3/5, 4/9) better.

Cons:

- KNN is slow at prediction for large datasets.
- PCA must be fitted carefully to avoid information loss.
- Softmax performance depends on good hyperparameter tuning.

Summary

In summary, the PCA + KNN + Softmax stacked pipeline combines the strengths of all three methods, leading to a more stable and dependable classifier for MNIST compared to using any single model alone.

2 Hyper-parameter tuning

Table 1: Hyperparameter Tuning Results for PCA + KNN + Softmax Stacked Classifier

#	PCA	KNN (k)	LR (lr, epochs)	Train time (s)	Train Acc.	Val Acc.	Train Recall	Val Recall	Train F1 / Val F1
1	100	5	0.2, 400	10.41	0.969306	0.955582	0.968825	0.955155	0.969101 / 0.955528
2	100	5	0.1, 200	10.40	0.967706	0.951180	0.967128	0.950511	0.967510 / 0.951189
3	100	7	0.1, 200	10.37	0.960908	0.953581	0.960277	0.952950	0.960659 / 0.953508
4	120	5	0.1, 200	11.48	0.967007	0.951180	0.966402	0.950571	0.966819 / 0.951161

Note: Train/validation metrics (Accuracy, Recall, F1) and training times were read from the provided terminal screenshots.

process of hyper parameter tuning. * I have started this stacked model after trying and failing to combine the xgboost and softmax and KNN as the primary learners and logistic regression as the eta learner.

* Then I tried to research on the optimal hyper parameters that we keep to make my current mode better.

* for each model the hyper parameters required for fine tuning is different cases.

* I started with the test run 1 with the parameters as shown, for the next run i hav ekept "k" same but changed the learning rate for the softmax and no.of epoches, as i found out that 200 epoches is good enough and learning rate 0.1.

* for the next try i tried to change the **K** in the KNN to 7 and kept the remaining same , but I havent observed , satisfactory results.

* Then I thought if I increase the no.of components then many be we can make a better model so I change the PCA to "120" but it still fails short to the 1st run.

so I decided to keep the 1st run.

3 Evaluation results on training and validation datasets

Table 2: Best Hyperparameter Configuration for PCA + KNN + Softmax Stacked Classifier

#	PCA	KNN (k)	LR (lr, epochs)	Train time (s)	Train Acc.	Val Acc.	Train Recall	Val Recall	Train F1 / Val F1
1	100	5	0.2, 400	10.41	0.969306	0.955582	0.968825	0.955155	0.969101 / 0.955528

This is the 1st run ,validation and Evaluation of the hyper parameters and model.

To optimize the system and reduce overall run time, I focused on a few practical steps. First, I applied PCA to reduce the 784-dimensional input to a smaller feature space, which made every model run considerably faster. I also tuned the KNN value of k , the softmax learning rate, and the number of training epochs so that the model reached good accuracy without unnecessary computation. Throughout the process, I regularly checked both training and validation metrics to avoid overfitting and to confirm that each change actually improved

performance. The final results showed that the selected configuration provided strong accuracy and F1 scores on both datasets while keeping the runtime manageable.

4 Summary of Thoughts and Observations

Thoughts

I tried various models and methods and finally i landed upon the stacked model , which uses the KNN as primary and softmax as the secondary.I was really frustated in my failed attempts to make a more complex model as a classifier and finally settle with my current code , I have tried different combinations of the models and methods , once basedon their inherent property and once based on their limitations and I believed mixed models or bagging models can give us a more reliable resultls as they give us a more process and stable predictions.

In this project, I built and evaluated a handwritten digit classification system using PCA for dimensionality reduction, KNN as a base classifier, and a softmax regression model as the meta-learner in a stacked architecture. I experimented with different hyperparameters to balance accuracy and computation time, and compared training and validation results to ensure the model generalised well. Overall, the project helped me understand how preprocessing, model selection, and careful tuning work together to create an efficient and reliable machine learning pipeline.